Chapter 5: Modelling with Classes

Definitions

UML and diagram types

- **Model:** A simplified representation of selected aspects of a system or domain.
- **UML (Unified Modelling Language):** A standardized graphical language for describing software structure and behaviour.
- **OMG (Object Management Group):** The standards organization responsible for UML and OCL specifications.
- **Class diagram:** A diagram showing classes, their attributes and operations, and structural relationships among them.
- **Object diagram:** A snapshot of particular objects, their attribute values, and the links among them.
- **Sequence diagram:** An interaction diagram emphasizing the order of messages exchanged by participants over time.
- **Communication diagram:** An interaction diagram emphasizing participant links and the messages exchanged along them.
- **State diagram:** A diagram showing states and the transitions between them in response to events.
- **Activity diagram:** A diagram showing flows of actions and control or data, including decisions and concurrent work.
- **Component diagram:** A diagram showing software components and their interfaces and dependencies.
- **Deployment diagram:** A diagram showing execution nodes and how software artifacts are placed on them.

Classes and structural relationships

- **Class:** A description of the common structure and behaviour of its instances, drawn as a named rectangle in UML.
- **Object / instance:** A particular entity with identity, state, and behaviour conforming to a class.
- **Attribute:** A property describing data held by an instance.
- **Operation:** A service or behaviour offered by a class, independently of its implementation.

- **Operation signature:** The name, parameters and their types, and return type describing an operation in the lecture's UML notation.
- **Association:** A structural relationship between instances of classes.
- **Link:** A particular instance of an association connecting objects.
- **Multiplicity:** The permitted number of objects at one association end that may be linked to one object at the opposite end.
- **Multiplicity notation:** `1` means exactly one; `0..1` means zero or one; `*` or `0..*` means zero or more; `1..*` means one or more.
- **Role name:** A name describing the function of an association end, such as supervisor or subordinate.
- **Reflexive association:** An association connecting a class to itself; its links can connect different instances of that class.
- **Symmetric relationship:** A relationship in which relating one object to another also implies the reverse relationship. Reflexive associations need not be symmetric.
- **Navigability:** The ability to access related objects through an association in a specified direction.
- **Bidirectional association:** An association supporting navigation in both directions.
- **Association class:** A class attached to an association to hold information or behaviour belonging to the relationship itself.
- **Intermediary / association object:** An object explicitly representing a relationship and linking its endpoints, such as an enrolment holding a student's grade in a course offering.
- **Generalization:** An is-a relationship between a specialized class and a more general class, shown with a hollow triangle pointing to the superclass.
- **Superclass:** A general class whose applicable features are inherited by subclasses.
- **Subclass:** A specialized class that inherits features from a superclass.
- **Generalization set:** A group of related generalizations representing a classification of a superclass.
- **Discriminator:** The criterion or classification dimension distinguishing specializations, such as account type.
- **Multiple inheritance:** Inheritance from more than one superclass, where supported by the modelling or programming language.
- **Role object:** A separate object representing a role an entity can acquire, lose, or hold alongside other roles without changing its own runtime class.

Whole-part relationships and interfaces

- **Shared aggregation:** A whole-part association shown with a hollow diamond at the whole end; its intended ownership semantics should be documented.
- **Composition:** A strong whole-part relationship shown with a filled diamond. A part belongs to at most one composite at a time, and parts still owned when the composite is destroyed are destroyed with it.
- **Composite / aggregate:** The whole object in a whole-part relationship.
- **Lifecycle:** The stages and rules governing an object's creation, existence, and destruction.
- **Propagation:** Implementing an operation on a whole by explicitly invoking corresponding operations on its parts.
- **Interface:** A contract of visible behaviour that implementing classes provide.
- **Realization:** A relationship indicating that a class implements an interface's contract, shown with a dashed line and hollow triangle toward the interface.
- **Provided interface:** An interface offered by a classifier or component, which UML can depict as a small circle.
- **Diagram note:** Explanatory text attached to a diagram to document assumptions, constraints, or rationale.
- **Constraint:** A rule limiting the valid states or behaviour of model elements.

Object Constraint Language

- **OCL:** Object Constraint Language; a language for expressing precise constraints, queries, and derived values on models without changing model state.
- **Side-effect-free expression:** An expression that evaluates a value without modifying the objects it examines.
- **Boolean constraint:** An expression that must evaluate to true for the model rule to be satisfied.
- **Context:** The type or operation to which an OCL expression applies.
- `self`**:** The contextual instance on which an OCL expression is evaluated.
- **Invariant:** A condition that must hold for instances in the valid states specified by the model.
- **Precondition:** A condition assumed to hold before an operation executes.
- **Postcondition:** A condition that must hold after an operation completes when its preconditions hold.
- `result`**:** The returned value referenced in an operation's OCL postcondition.

- **Dot navigation:** Accessing an object's properties or query operations using a dot, as in `self.age`.
- **Collection navigation:** Applying operations to a collection using arrow notation. Example: `self.employee->size()`.
- `Set`**:** An unordered collection containing no duplicates.
- `OrderedSet`**:** An ordered collection containing no duplicates.
- `Bag`**:** An unordered collection that permits duplicates.
- `Sequence`**:** An ordered collection that permits duplicates.
- **Logical operators:** `and` requires both conditions; `or` requires at least one; `not` negates a condition; `xor` requires exactly one; `implies` requires the consequent whenever the antecedent is true.
- **Equality and inequality:** OCL uses `=` to test equality and `<>` to test inequality.
- `isEmpty() / notEmpty()`**:** Tests whether a collection contains no elements or at least one element, respectively.
- `size()`**:** Returns the number of elements in a collection.
- `includes(x) / excludes(x)`**:** Tests whether a collection contains or does not contain an element.
- `sum()`**:** Computes the total of a collection's numeric elements.
- `exists`**:** Tests whether at least one collection element satisfies a condition.
- `forAll`**:** Tests whether every collection element satisfies a condition; it is true for an empty collection.
- `select / reject`**:** Returns a collection containing elements that satisfy or do not satisfy a condition, respectively, without modifying the original collection.
- `includesAll(c) / excludesAll(c)`**:** Tests whether all elements of another collection are included or none are included, respectively.
- **Set intersection:** A set operation returning elements common to both sets.
- **Set union:** A set operation returning elements belonging to either set.
- `excluding(x)`**:** Returns a collection with the specified element excluded.
- `first()`**:** Returns the first element of a nonempty ordered collection.
- `oclIsUndefined()`**:** Tests whether a value is undefined, including null or invalid; the worked example uses it to check for an absent supervisor.

- `allInstances()`: Refers to the instances of a type in the model's evaluation scope.
- `isUnique`: Tests whether an expression produces a different value for each collection element.
- **Guard:** A condition controlling when another condition or action applies, such as excluding self-comparisons in a uniqueness rule.

Developing and implementing models

- **Exploratory domain model:** A model used to understand concepts and relationships in the problem domain.
- **System domain model:** A model of domain information that the application will represent.
- **Complete system model:** A model that includes domain classes together with interface, architecture, and utility classes needed by the system.
- **Domain logic:** Rules and operations of the problem domain, kept independent of a particular user interface.
- **Candidate class:** A possible class considered during analysis and retained only if it has an appropriate role in the model.
- **Responsibility:** Work that the system must perform and that is assigned to suitable collaborating classes.
- **Collaboration:** Objects working together to fulfil responsibilities through operations and relationships.
- **Persistence:** Retaining information beyond the lifetime of a running process, commonly in files or a database.
- **Design pattern:** A reusable description of a solution structure for a recurring design problem.
- **Iterative modelling:** Repeatedly refining a model as scenarios, requirements, and understanding reveal improvements.
- **Field:** A variable declared within a class, used to implement an attribute or relationship in Java.
- `extends`: The Java keyword establishing class inheritance.
- `implements`: The Java keyword declaring that a class provides an interface's contract.
- **Inverse link:** The corresponding reference or collection entry at the opposite end of a bidirectional relationship.
- **Constructor:** Code that initializes a newly created object and helps establish a valid initial state.
- **Factory method:** An operation responsible for creating and returning an object, allowing initialization and link creation to be controlled.

- **Unmodifiable snapshot:** A copy of collection contents that callers cannot change and that does not reflect later structural changes to the original collection.
- **Reassignment:** Moving an association link to another object while consistently updating the old inverse link, new inverse link, and reference.

5.0 UML and Diagram Types

The Unified Modelling Language (UML) provides graphical notation for modelling software. The lecture credits Booch, Rumbaugh, and Jacobson and describes OMG standardization beginning in 1997.

- Class diagrams show classes and structural relationships.
- Sequence and communication diagrams show interactions among objects.
- State and activity diagrams describe behaviour and internal flows.
- Component and deployment diagrams show logical organization and physical placement.

5.1 Essentials of Class Diagrams

- A class is a named rectangle, optionally divided into attribute and operation compartments.
- Attributes describe simple data. Associations describe relationships between instances. Generalizations describe inheritance.
- Operation signature: operationName(parameterName: ParameterType): ReturnType.
- Show details appropriate to the model's purpose. Domain models need not contain every implementation method.

5.2 Associations and Multiplicity

An association describes a structural relationship between instances. A link is one instance of that relationship.

5.2.0 Reading multiplicity

Read the multiplicity at one end as the number of objects at that end that may be linked to one object at the opposite end.

- 1 means exactly one; 0..1 means optional; * or 0..* means zero or more; 1..* means one or more. A range gives explicit lower and upper bounds.
- Employee to Company: each employee works for one company in the example's scope; a company can have zero or more employees.
- Company to BoardOfDirectors: each company has exactly one board and each board belongs to exactly one company.
- Passenger to Booking: each booking belongs to exactly one passenger; a passenger can have zero or more bookings.

- Validate both directions, zero cases, and temporary states. Multiplicity expresses domain rules rather than just typical data.

5.2.1 Names, roles, and navigation

- Clear association and role names explain the relationship and distinguish its ends.
- Reflexive associations connect a class to itself, such as Employee supervising Employee. Roles distinguish supervisor and subordinate.
- Reflexive does not imply symmetric: supervising someone does not mean they supervise you.
- Navigation indicates which objects can access related objects. The lecture uses arrows to restrict direction.
- The slides treat undecorated associations as bidirectional; general UML usage can leave navigability unspecified. State the convention used in a model.

5.2.2 Association classes

- Use an association class when information belongs to the relationship rather than either endpoint independently.
- Study example: Enrolment links Student and CourseOffering and holds the grade for that enrolment. The grade is not an intrinsic property of either endpoint alone.
- UML connects an association class to its association using a dashed line. An explicit intermediary class can also represent relationship information.
- Avoid unnecessary one-to-one classes when the information has no distinct identity, lifecycle, or responsibility.

5.3 Generalization

- A hollow triangular arrow points from subclass to superclass. Apply the is-a test and check that inherited behaviour remains meaningful.
- A generalization set groups specializations using a discriminator, the criterion distinguishing subclasses.
- Do not turn individual objects or simple attribute values into subclasses without a structural or behavioural reason.
- Multiple classification criteria may suggest additional generalization levels, multiple inheritance where supported, or separate role objects.

- When a person's role can change, separate the role rather than requiring an existing object to change its runtime class.

5.3.0 Choosing inheritance or a role

A SavingsAccount can specialize Account when every savings account supports the account contract. In contrast, Student and Employee may be changing roles of a Person. A person can hold both roles, so separate role objects can express this without changing the Person object's class.

- A discriminator explains a classification dimension, such as account type. Multiple dimensions need separate consideration rather than a hierarchy containing every possible combination.
- Generalization expresses what an object is. An association expresses what it knows, belongs to, or is linked to.

5.4 Object Diagrams

- Object diagrams show snapshots of particular instances, attribute values, and links.
- An underlined label commonly has the form objectName:ClassName; an unnamed instance can use :ClassName.
- Links instantiate associations. Class-level generalization arrows do not appear between objects.
- A subclass instance is also an instance of its superclasses. It does not require a separate superclass object linked to it.

5.5 Aggregation, Composition, and Interfaces

5.5.0 Whole-part relationships

- Shared aggregation uses a hollow diamond at the whole end. Document the intended ownership meaning.
- Composition uses a filled diamond at the whole end. A part belongs to at most one composite at a time; parts still owned by a destroyed composite are destroyed with it.
- Holding a reference alone does not imply composition. Consider ownership and lifecycle rules.
- Propagation occurs when an aggregate implements an operation by calling corresponding operations on its parts. It requires explicit programming, unlike language-supported inheritance.

5.5.1 Interfaces and explanatory notes

- An interface specifies visible behaviour shared by implementing classes, useful for a common contract among unrelated classes.
- UML realization uses a dashed line with a hollow triangle toward the interface; a small circle can represent a provided interface.
- Diagram notes and surrounding prose explain assumptions, constraints, and rationale that graphics alone cannot express clearly.

5.6 Object Constraint Language (OCL)

OCL expresses precise, side-effect-free properties of a model. Boolean constraints express rules that must hold; other OCL expressions can calculate values or navigate objects without changing state.

5.6.0 Context and conditions

- context identifies the type or operation being constrained; self denotes the contextual instance.
- Invariants hold for instances in valid states described by the model.
- Preconditions are assumed before an operation. Postconditions state what must hold afterward; result denotes the returned value.
- Example: context Person inv: self.age >= 0.
- Slide example: context Person::income(): Integer pre: self.age >= 18 post: result < 5000. These are illustrative domain rules, not universal facts about income.

5.6.1 Navigation and types

- Dot navigation accesses properties and query operations.
  Examples: `self.age`, `self.manager`, and `self.stockPrice()`.
- A many-valued role evaluates to a collection. Use `->` for its operations.
  Example: `self.employee->size()`.
- Set is unordered with unique elements; OrderedSet is ordered with unique elements. Bag permits duplicates; Sequence is ordered and permits duplicates.
- The lecture gives both Sequence and OrderedSet descriptions for ordered roles. The appropriate collection type also depends on whether duplicates are permitted.
- Operators include and, or, not, xor, implies, =, and <>. Arithmetic and string expressions can appear within constraints.

5.6.2 Collection operations

- isEmpty() and notEmpty() test for elements; size() counts them; includes(x) and excludes(x) test membership; sum() totals numeric elements.
- exists(x — condition) requires at least one element satisfying the condition.
- forAll(x — condition) requires every element to satisfy it. It is true for an empty collection, so also require notEmpty() when at least one element is necessary.
- select(x — condition) retains matching elements; reject(x — condition) removes them.
- includesAll(c) and excludesAll(c) compare membership against another collection.
- Set intersection keeps common elements, union combines elements, and excluding(x) removes an element. first() accesses the first element of a nonempty ordered sequence.

5.6.3 Example constraints

- context Company inv: self.employee->notEmpty() requires at least one employee.
- context Company inv: self.employee->select(p — p.age > 50)->notEmpty() requires at least one employee older than 50.
- context Company inv: self.employee->forAll(p — p.forename = 'Jack') requires every employee's forename to be Jack.
- context Company inv: self.employee->forAll(e1, e2 — e1 <> e2 implies e1.forename <> e2.forename) requires distinct employees to have distinct forenames. The guard excludes self-comparison.

5.6.4 Practice rules from the lecture

- Supervisor age and salary comparisons must account for an absent supervisor when the relationship is optional.
- A department manager must belong to that department's employees; employee identifiers must be unique.
- A project's location must be among its department's locations, and an employee cannot supervise itself.
- To limit workers below an hours threshold to two, select those work assignments and require the selected collection's size to be at most two. Hours belong to the work relationship when employees work on multiple projects.

5.6.5 Worked solutions to the practice rules

The following solutions use explicit example role names: Employee has an optional supervisor, age, salary, and ssn; Department has employees, manager, and locations; Project has department, location, and assignments; each WorkAssignment has employee and hours. Match these names to the roles in the class diagram before applying the constraints.

When a supervisor exists, the supervisor must be older and earn at least as much. The conditional avoids navigating an absent supervisor.

```
context Employee inv SupervisorAge:
  if self.supervisor.oclIsUndefined() then true
  else self.supervisor.age > self.age endif

context Employee inv SupervisorSalary:
  if self.supervisor.oclIsUndefined() then true
  else self.salary <= self.supervisor.salary endif

context Employee inv NoSelfSupervision:
  if self.supervisor.oclIsUndefined() then true
  else self.supervisor <> self endif
```

Assuming every department has exactly one manager, the manager must be included in its employee collection. Employee identifiers must be unique over the model's employee population.

```
context Department inv ManagerIsEmployee:
  self.employees->includes(self.manager)

context Employee inv UniqueSSN:
  Employee.allInstances()->isUnique(e | e.ssn)
```

The identifier rule assumes ssn is required. Uniqueness and presence are different requirements: uniqueness alone does not establish that each employee has a valid identifier.

A project's location must belong to its department's locations. At most two employees may work fewer than ten hours on that project. Here each employee has at most one assignment per project, so counting assignments also counts employees.

```
context Project inv AllowedLocation:
  self.department.locations->includes(self.location)

context Project inv OneAssignmentPerEmployee:
  self.assignments->isUnique(a | a.employee)

context Project inv AtMostTwoShortAssignments:
  self.assignments->select(a | a.hours < 10)->size() <= 2
```

If the model permits several assignments per employee on one project, first define whether the ten-hour limit applies to an individual assignment or the employee's total project hours. The constraint must follow that domain rule.

5.7 Developing Class Diagrams

5.7.0 Model scope

- An exploratory domain model helps understand the field.
- A system domain model captures domain information represented by the application.
- A complete system model adds UI, architecture, and utility classes. Keep domain logic independent of a particular UI.

5.7.1 Iterative process

- Identify candidate classes, starting with one or two central ones.
- Add clear attributes and associations, specify multiplicities, and identify useful generalizations.
- Expand to other classes, allocate responsibilities, and identify operations and interfaces.
- Review and iterate, adding or removing elements as understanding improves. Consider reuse and relevant design patterns.

5.7.2 Finding classes and relationships

- Extract nouns and noun phrases from requirements; reject duplicates, instances, vague concepts, and information outside scope.
- Some rejected nouns become attributes. Simple values usually belong in attributes; entities with identity and relationships may require classes.
- Associations describe structural relationships such as membership, ownership, or connection.
- Actions generally become operations. In a library, borrow creates a Loan, while return changes its returned date; these actions should not simply become permanent associations.
- Avoid duplicate attributes. A coherent group of information may justify a separate class.
- Find generalizations upward from similar classes or downward from general concepts. Prefer an interface for a small shared contract among otherwise different classes.

5.7.3 Responsibilities and operations

- A responsibility is work the system must perform. Assign each functional requirement to appropriate collaborating classes.
- Keep responsibilities related. Split overloaded classes, reconsider classes with no responsibilities, and add a class when no existing one fits a required responsibility.

- Responsibilities include creation, attribute access, persistence, linking, searching, computation, transformation, and output.
- Use cases and action verbs help identify responsibilities. Paper cards can record names, attributes, and responsibilities while relationships are rearranged.
- Public operations provide needed services; supporting methods use the narrowest practical visibility.

5.7.4 Collaboration patterns

- Adding a bidirectional link updates both objects consistently.
- Creating a linked object initializes its link and updates the existing object's reference or collection.
- Creating an association object connects it to both endpoints and updates their inverse links.
- Reassigning a link removes the old inverse link and establishes the new one.
- Searching an association traverses related objects and compares the relevant property.

5.8 Implementing Class Diagrams in Java

- Attributes become fields; generalization becomes extends; realization becomes implements.
- A 1 or 0..1 association end commonly becomes an object reference; a many-valued end commonly becomes a collection.
- References do not automatically enforce required multiplicity; collections do not automatically enforce minimum size or uniqueness. Constructors and update methods must preserve constraints.
- Bidirectional relationships usually need fields at both ends and controlled updates to keep them consistent.
- In the lecture example, RegularFlight.addSpecificFlight creates a SpecificFlight referring back to that RegularFlight, then adds it to the collection.
- Restrict low-level constructors and helper operations so callers cannot create invalid partial relationships.

5.8.0 Worked Java example: a one-to-many association

This study example implements the lecture's RegularFlight–SpecificFlight relationship. A regular flight may have zero or more specific flights, and each specific flight belongs to exactly one regular

flight. The factory method establishes both directions. The private constructor prevents clients from bypassing that method.

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public final class RegularFlight {
    private final List<SpecificFlight> flights = new ArrayList<>();

    public SpecificFlight addSpecificFlight(LocalDate date) {
        SpecificFlight flight = new SpecificFlight(date, this);
        flights.add(flight);
        return flight;
    }

    public List<SpecificFlight> getSpecificFlights() {
        return List.copyOf(flights);
    }

    public static final class SpecificFlight {
        private final LocalDate date;
        private final RegularFlight regularFlight;

        private SpecificFlight(LocalDate date,
                               RegularFlight regularFlight) {
            this.date = Objects.requireNonNull(date);
            this.regularFlight =
                Objects.requireNonNull(regularFlight);
        }

        public LocalDate getDate() { return date; }

        public RegularFlight getRegularFlight() {
            return regularFlight;
        }
    }
}
```

- The single reference enforces one owner after construction; the collection supports many specific flights. An empty collection satisfies the zero lower bound.
- The returned list is an unmodifiable snapshot, so callers cannot remove links through the getter.
- This example keeps the owner fixed. If reassignment is required, an operation must update the old owner's collection, the new owner's collection, and the reference together.
- Required references do not by themselves imply UML composition. Composition additionally requires the model's ownership and destruction rules.

5.9 Difficulties and Risks

- Strong programming skills do not automatically produce good domain models; choosing abstractions and responsibilities is a separate skill.
- Train the team, involve an experienced modeller, and review diagrams against requirements and concrete scenarios.
- Check multiplicities, role names, inheritance, lifecycle assumptions, and whether implementation operations preserve constraints.